

14.5. The Turing Machine

The PDA is still not powerful enough to solve all of the problems that can be solved by a digital computer. An example of one type of problem that a PDA cannot solve is recognizing palindromes of arbitrary length. A pushdown machine would need to know when it has reached the middle of an input string in order to be able to recognize palindromes, but it would need to detect a specially inserted character to know when that point had been reached. The situation of a PDA reading a tape that either advances or remains stationary is analogous to an observer watching a train emerge from a tunnel. Neither the machine nor the observer has any means of locating the middle without a special character or specially marked boxcar providing an internal clue.

Before discussing ways of extending the capability of the PDA, we will briefly digress and design a machine that is able to recognize palindromes containing a special character that indicates the location of the midpoint of the string. Palindromes with a midpoint indicator are strings of the form wcw_r where w_r is the reverse of substring w and c is the special midpoint marker. Let the input alphabet for this machine consists of the set

$$\Sigma = \omega \cup \{c, \#\}$$

where ω is the set of all admissible input characters except the midpoint marker, c , and the end marker, $\#$. The set of admissible characters will depend upon the application, but may be the characters in the alphabet, the digits, the DNA bases $\{C, G, H, T\}$, the bits $\{0, 1\}$, or some other finite set of symbols. The stack alphabet consists of ω and the bottom marker ∇ .

Control requires two intermediate states for processing the input, and final states signifying acceptance or rejection of the input string. In figure 14.12 below, the PDA remains in state q_0 until the midpoint marker is detected. While in state q_0 every new input character is pushed onto the stack. When the midpoint marker is detected, the machine makes a transition to state q_1 . In this state each input character is compared to the character on the top of the stack. If they are identical, the tape is advanced, the stack is popped, and the machine remains in state q_1 . If the two characters are not identical, the machine makes a transition to the rejecting state q_3 . The string is recognized as a palindrome only when the end marker appears on the tape and the bottom marker is on the top of the stack.

In figure 14.16, the state transitions are moderated by the tape and stack symbol pair. We use the notation $(x,-)$ to indicate that the tape symbol is any character in the admissible set ω and that the stack symbol is irrelevant. A transition from state q_0 to state q_1 occurs whenever the midpoint marker is read by the tape head, regardless of the character on the top of the stack. The notation (x,x) is used to indicate when the tape head and stack symbols are identical (where x is any element of ω), and the notation (x,y) indicates that the tape and stack symbols differ. When these two symbols differ, a transition to the rejecting state, q_3 , occurs. Failure to detect the midpoint symbol will also lead to the string being rejected. Once the rejection state is entered, any additional input will be ignored, and the machine will remain in q_3 . The palindrome is not accepted until

the end marker is read by the tape head when the stack is empty (as indicated by the appearance of the bottom marker on the top of the stack.)

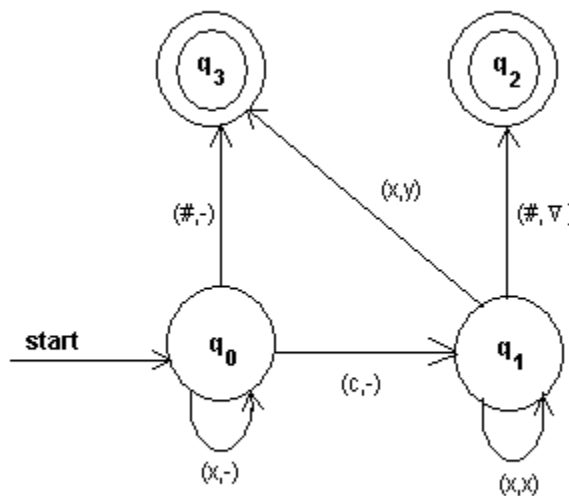


Figure 14.16 PDA for recognizing the strings wcw_r

Even with the artifice of marking the middle of the input string, we are only able to recognize palindromes that contain an even number of characters from the admissible set. Odd length strings such as *radar* have a character from the admissible alphabet at their midpoint. Though we may not consider recognizing palindromes as a significant problem, one for which we would be willing to shell out more money to buy a more sophisticated machine with this additional capability, the inability to locate the midpoint of an input string is a significant limitation of the PDA. It is this limitation that makes solving the general palindrome recognition problem on a PDA infeasible, and this limitation is indicative of a larger shortcoming of the PDA. It cannot count!

The inability to count to an arbitrary number handicapped the FSM, and it is also a shortcoming of the PDA. The PDA is an improvement on the FSM because it is able to recognize matched pairs of arbitrary length and recognize strings that conform to the rules of certain grammars, but it does not overcome the FSM's fundamental weakness. In order to determine what features need to be included in a simple computing machine with more power than a PDA, we will examine the properties needed by a machine that can recognize palindromes and recognize strings of the form $0^n 1^n$ where n is arbitrary.

First, let us describe an algorithm for recognizing palindromes. Assume for simplicity that the input alphabet consists only of the digits 0 and 1, and the end marker, #. The input tape will consist of the candidate string bracketed by the end marker symbols. We begin with the tape head positioned over the end marker at the left of the string. We progress by moving the tape one square to the left and examining the first character. We then "remember" whether this character is a 0 or a 1 and replace it with a new end marker. Now we move the tape forward until we observe the rightmost character in the string. This is the character immediately preceding the next end marker

that we encounter; so, we will move the tape to the left until we encounter the end marker and then back it up by one square. If this character is the same as the one we "remember", we replace it with an end marker and back the tape up (move it to the right) until we encounter an end marker at the front of the string. We then repeat the previous steps. If the character is different from the one "remembered", we reject the candidate and halt.

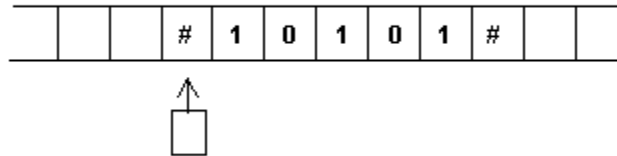


Figure 14.17 Beginning position for palindrome recognizing machine

The algorithm ends when the machine halts either because all of the input characters have been replaced and a palindrome recognized or the candidate has been rejected.

The machine can "remember" a character either by pushing it on a stack for subsequent recall or by making a state transition based upon an input character. With a finite, predetermined input alphabet, the latter approach can be implemented with a finite set of states. For the binary alphabet in our example, the palindrome recognition machine will require 8 states:

- q_0 start state, looking for the first input character
- q_1 remembering a 0, looking for an end marker
- q_2 remembering a 1, looking for an end marker
- q_3 remembering a 0 having found an end marker, looking for a match
- q_4 remembering a 1 having found an end marker, looking for a match
- q_5 returning to the front of the string after a successful match
- q_6 mismatch detected, final rejection state
- q_7 if input character at tape head is #, accept, else same as start state

The machine we have begun to describe differs from the PDA in two important ways: the tape is allowed to move in both directions and the tape can be written upon as well as read. Every move of the machine is determined by the current input character and the current state, and each move results in the machine being in a new state (which may be the same as the current one), a new symbol having been written on the tape (which again can be the same as the current symbol), and the tape head having been advanced one position to the left or right. The control of the machine is thus fully specified by the following set of transitions where we use the notation

(state, input symbol) $\Rightarrow$ (new state, new symbol, direction tape head is moved).
(Note! Rather than moving the tape, we will move the tape head. Moving the tape one square to the left is equivalent to moving the tape head one position to the right.)

$$(q_0, \#) \Rightarrow (q_0, \#, R)$$

$$(q_0, 0) \Rightarrow (q_1, \#, R)$$

$(q_0, 1) \Rightarrow (q_2, \#, R)$
 $(q_1, \#) \Rightarrow (q_3, \#, L)$
 $(q_1, 0) \Rightarrow (q_1, 0, R)$
 $(q_1, 1) \Rightarrow (q_1, 1, R)$
 $(q_2, \#) \Rightarrow (q_4, \#, L)$
 $(q_2, 0) \Rightarrow (q_2, 0, R)$
 $(q_2, 1) \Rightarrow (q_2, 1, R)$
 $(q_3, 0) \Rightarrow (q_5, \#, L)$
 $(q_3, 1) \Rightarrow (q_6, \#, \text{HALT})$
 $(q_3, \#) \Rightarrow (q_7, \#, \text{HALT})$
 $(q_4, 0) \Rightarrow (q_6, \#, \text{HALT})$
 $(q_4, 1) \Rightarrow (q_5, \#, L)$
 $(q_4, \#) \Rightarrow (q_7, \#, \text{HALT})$
 $(q_5, \#) \Rightarrow (q_7, \#, R)$
 $(q_5, 0) \Rightarrow (q_5, 0, L)$
 $(q_5, 1) \Rightarrow (q_5, 1, L)$
 $(q_7, \#) \Rightarrow (q_7, \#, \text{HALT})$
 $(q_7, 0) \Rightarrow (q_1, \#, R)$
 $(q_7, 1) \Rightarrow (q_2, \#, R)$

Suppose that after a number of moves the machine initially described in figure 14.17 has arrived in a configuration described by figure 14.18 below. The first and last two characters of the candidate string have been compared and found identical.

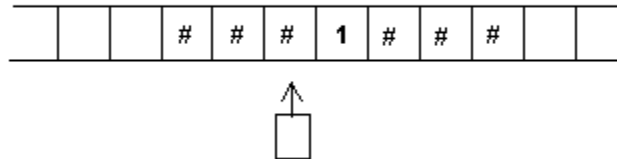


Figure 14.18 Palindrome recognizing machine after a sequence of moves

The machine, as pictured in figure 14.18, will be in state q_5 , having just completed its return to the beginning of the remaining string. The machine is now prepared to test a 1-character string, which by definition is a palindrome. The machine undergoes the following sequence of transitions terminating with it halted in an accepting state.

$(q_5, \#) \Rightarrow (q_7, \#, R)$
 $(q_7, 1) \Rightarrow (q_2, \#, R)$
 $(q_2, \#) \Rightarrow (q_4, \#, L)$
 $(q_4, \#) \Rightarrow (q_7, \#, \text{HALT})$

In the final transition, the machine is looking for a matching 1 on the tape, but when it finds an end marker, it recognizes that it is validating a 1-character substring and halts in an accepting state.

In the second example we are required to design a machine that accepts strings of the form $0^n 1^*$ for some arbitrary n . We must be able to count to n , but first we have to be told what value n is to have. This is accomplished by writing both the value of n and the string to be verified on the input tape. The value of n is most easily expressed in unary notation. Unary might be described as the preferred notation of Mr. Ed, the equine star of a semi-popular 1960's television show. (Mr. Ed is fondly regarded today as a TV classic, an honor that is often reserved for shows whose banality distinguishes them from the insipid background fare from which they emerge.) To express a number such as 21 in unary one simply writes 21 ones. It is not necessary to use a unary numbering system in our machine, but it is sufficient.

The algorithm for recognizing strings of the form $0^n 1^*$ is to simply cross out a 0 and decrement the count until both the count and the number of 0's at the front of the candidate string is zero, then make sure no further 0's appear before an end marker is located. Decrementing a unary number is achieved by removing a 1.

Let the tape alphabet consist of the characters 0, 1, #, and the blank character, b. End markers will frame the two input terms which are separated by blanks. Consider, for example, the case where $n = 4$.

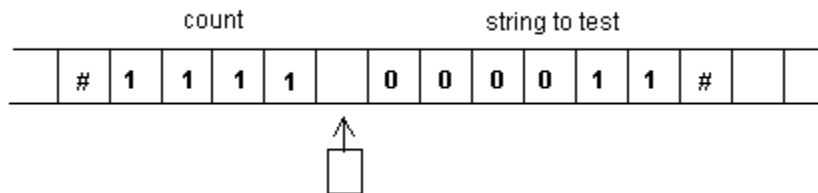


Figure 14.19 The initial state of a machine verifying $0^4 1^*$

The tape head will cross back and forth, ignoring blanks, canceling first a 1 and then a 0 with a blank until the left end marker is located. Then, all that should be left between the tape head and the right end marker are blanks and zero or more 1's. If a 0 character appears before the final end marker, the candidate string is rejected. The machine we have just described has four states:

- q_0 the state of moving left, ignoring blanks, looking for a 1 (start state)
- q_1 the state of moving right, ignoring blanks, looking for a 0
- q_2 the state of having located the left end marker, moving right, ignoring 1's, looking for the right end marker. (the accepting state)
- q_3 the rejecting state

The state transitions for this machine are;

$$(q_0, b) \Rightarrow (q_0, b, L)$$

$$\begin{aligned}
(q_0, 0) &\Rightarrow (q_3, 0, \text{HALT}) \\
(q_0, 1) &\Rightarrow (q_1, b, R) \\
(q_0, \#) &\Rightarrow (q_2, \#, R) \\
(q_1, b) &\Rightarrow (q_1, b, R) \\
(q_1, 0) &\Rightarrow (q_0, b, L) \\
(q_1, 1) &\Rightarrow (q_3, 1, \text{HALT}) \\
(q_1, \#) &\Rightarrow (q_3, \#, \text{HALT}) \\
(q_2, b) &\Rightarrow (q_2, b, R) \\
(q_2, 0) &\Rightarrow (q_3, 0, \text{HALT}) \\
(q_2, 1) &\Rightarrow (q_2, 1, R) \\
(q_2, \#) &\Rightarrow (q_2, \#, \text{HALT})
\end{aligned}$$

Figure 14.20 Listing of the transition functions for a TM accepting 0^n1^*

The machine we have described is able to count by reading and decrementing a supplied value on its input tape. The tape serves as both a vehicle for supplying input data and a scratch memory for recording intermediate results. The tape head is able to advance in either direction subject to the control of the machine. This machine was first described by Alan Turing and is named for him. The Turing machine is described formally by the following septuplet:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, b, F)$$

where

Q is a finite set of states

Γ is the finite set of allowable tape symbols

b is the blank symbol, an element of Γ

Σ is a subset of Γ not including b , the set of input symbols

δ is a mapping from $Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$, the transitions

q_0 is the start state, $q_0 \in Q$

F is the set of final states, $F \subseteq Q$.

A Turing machine has finite state control and an external memory, the infinite tape, capable of remembering infinitely long sequences of input symbols. In addition to being able to accept well-formed strings of a language such as the set of palindromes over the alphabet $\{0, 1\}$, a Turing machine is able to compute functions that map integers into integers.

In section 14.1 of this chapter we have built an FSM capable of performing sequential addition on a pair of input strings. That machine only needed to "remember" whether the last sum produced a carry or not to be able to process the next input pair. For multiplication, however, arbitrarily long intermediate results must be retained and added to the new partial product for each digit (or bit) in the multiplier. The Turing machine provides the necessary tape for storing intermediate results and the ability to move the tape head in both directions to access different storage areas that is needed to perform multiplication.

A Turing machine for multiplying two unary positive integers can be constructed with an input alphabet that includes the binary digits 0 and 1, an end marker, #, and an operation indicator, *, that separates the multiplicand and multiplier. The tape alphabet will also contain blank symbols, b. This machine will need six states for control. Alternative Turing machines with fewer tape symbols and additional states can also be designed for performing the same multiplication function.

Initially, the TM will have the tape head located at the symbol * that separates the multiplicand and the multiplier. The input is separated from the rest of the tape by end markers, and the remainder of the tape contains blank symbols.

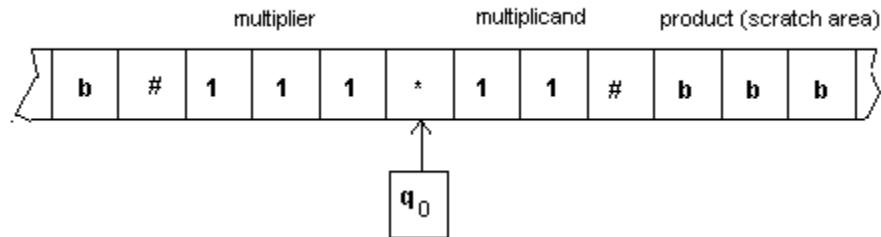


Figure 14.21 Initial configuration of a TM for multiplying unary integers

The product will be written to the right of the input in a section of the tape that is now blank. The algorithm for performing the multiplication begins by moving the tape head to the left until a 1 is read. The 1 is changed to 0 and the tape head is moved right to copy each bit of the multiplicand into the blank cells of the scratch area. The copy phase is accomplished by converting each bit of the multiplicand to 0 on the initial pass and then repeatedly returning to replace a 0 in the multiplicand with a 1 followed by a move to the right to copy a 1 into the first blank cell encountered in the scratch area. Each time the entire multiplicand is copied to the scratch area the whole process repeats until all of the 1's in the multiplier have been converted. The state transitions governing the operation of this machine are given in the table shown in figure 14.22 below.

States	Input symbols				
	0	1	#	*	b
q ₀	q ₀ ,0,L	q ₁ ,0,R	q ₅ , halt	q ₀ ,*,L	
q ₁	q ₁ ,0,R	q ₁ ,0,R	q ₂ ,#,L	q ₁ ,*,R	
q ₂	q ₃ ,1,R	q ₂ ,1,L			
q ₃		q ₃ ,1,R	q ₃ ,#,R		q ₄ ,1,L
q ₄	q ₃ ,1,R	q ₄ ,1,L	q ₄ ,#,L	q ₀ ,*,L	

Figure 14.22 Table of state transitions for the multiplier TM

The empty cells in this table represent state/ input symbol pairs that cannot occur. The system begins in state q₀ and remains in this state searching to the left for the first occurrence of a 1 in the multiplier. When a 1 is found, it is replaced with a 0 and the system makes a transition to state q₁. In state q₁ the machine moves all the way to the end of the multiplicand string converting every 1 it finds to a 0. (The only 1's that will be

accessed by the machine in state q_1 are those in the multiplicand.) Once the right end marker is reached, the TM makes a transition to state q_2 and moves back to the left searching for a 0. During the copying phase of its operation, the system will alternate between state q_3 (moving right until a blank is found in the scratch area of the tape) and state q_4 (returning to the left to find the next 0 in the multiplicand). The copy operation will complete in state q_4 when no more 0's are left in the multiplicand and the separator symbol is reached. At this point the system returns to state q_0 and the process is repeated for the next bit in the unary multiplier. The multiplication operation terminates when the left end marker is encountered during a search for a 1 in the multiplier. At termination the product in unary notation appears to the right of the input strings.

The Turing machine is a very powerful construct. Every algorithm that can be computed on a digital computer can be described as a sequence of mechanical operations performed by a Turing machine. The tape provides both an input/output device and storage capacity. The infinite capacity of the tape accommodates the requirements for problems of any finite size. The bi-directional motion of the tape head allows program controlled access to data strings in any segment of the tape.

While a digital computer has the ability to solve a variety of problems, the Turing machines that we have described so far are constructed to solve a single problem. The Turing machine that recognized binary palindromes had a different set of states and possibly a different tape alphabet than the Turing machine for multiplying positive integers. To achieve the full power of the digital computer, we will need to build a machine that is capable of recognizing a set of instructions that describe the operation of a Turing machine along with the input string it is to evaluate. Such a machine is called a Universal Turing machine.

Problem-specific machines are not confined to the realm of formal computational methods. In the early days of the digital computer, some machines were built to solve a single, computationally intensive problem. John MacDonald, a senior engineer with IBM and later Professor of Computer Science at Marist College, once related an anecdote to me about an experience he had when he was working for IBM. There was a need to solve a particular differential equation that nobody could solve analytically, so a team of mathematicians and engineers was assembled to design a machine for solving the problem numerically. When the design was complete the team decided to show their work to John von Neumann, the brilliant mathematician and computer pioneer who regularly consulted with IBM, before proceeding further. They met with von Neumann and he agreed that the machine could be built, but asked why they would want to build it. When they explained the problem to him, he exclaimed, "But the solution to that equation is well known!" Von Neumann proceeded to produce an analytical solution, and the machine was never built.

To illustrate the principles used in building a Universal Turing machine, we will construct a TM with three parallel tapes and three separate tape heads and present it with a program for recognizing strings of the form $0^n 1^*$. Any language that is accepted by a multi-tape machine will also be accepted by a single-tape machine, but the multi-tape machine is easier to describe and better illustrates the features of a machine that uses programmed control. The three tapes will be used for control information, input data, and scratch memory respectively.

The three tape machine will start with the program tape head located at the start state, the data tape head located at the initial position shown in figure 14.21 above, and the scratch tape empty. The program tape consists of the set of transitions for all state/input symbol pairs with a separator character delineating each of the individual transitions. The machine writes the initial state and data symbol on the scratch tape, and then searches the program tape for a match. When a match is found, the new state is written on the scratch tape, the new symbol is written on the data tape and the scratch tape, and the data tape head is moved in the indicated direction. This process repeats until the program halts.

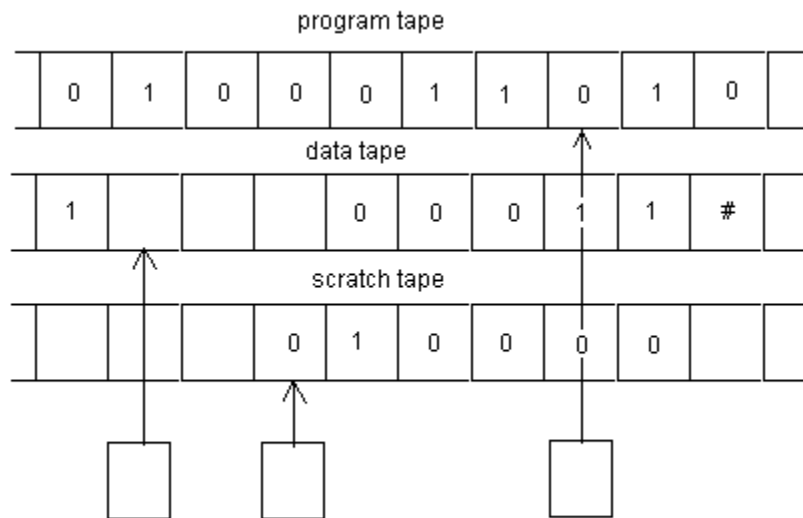


Figure 14.23 A three tape UTM

A Universal Turing machine (UTM) executes programs. Running a program is an effective procedure that can be decomposed into a series of mechanical operations. The programs that run on a Universal Turing machine contain the instructions for running the problem-specific Turing machines like those we have previously described. In order for the Universal Turing machine to be able to run a program, that program must be formulated in a language recognized by the UTM. We will call this language the Universal Language.

In the problem-specific examples, we saw that the set of tape symbols and states was not unique. A variety of Turing machines could be constructed to solve the same problem, just as a variety of architectures and instruction sets could be employed to construct digital computers with the same capability. The number of states needed by each problem-specific Turing machine varies, but is finite and unbounded. Therefore, in the Universal Language we must have a unique tape alphabet, and we must limit the symbols for indicating a state but not limit the number of states that can be represented.

In our Universal Language, we will restrict the tape alphabet to the set $\{0, 1, \#, b\}$. The data tape will contain just these symbols. The program tape must express state transition functions of the form $(q_0, 1) \rightarrow (q_1, b, R)$. Each state transition function

contains five pieces of information: the current state, the current input symbol, the new state, the new input symbol, and the direction in which the (data) tape head is to move. These five pieces of information must be individually recognizable, and each transition function must be delineated from the rest. The program tape must also use a fixed set of symbols. These requirements are satisfied if we use a code where we number the states, input tape symbols, and directions of motion and express these numerical values in unary notation. For instance the states will be numbered as follows:

$$\begin{aligned} q_0 &= 0 \\ q_1 &= 00 \\ q_2 &= 000 \end{aligned}$$

where we use 1, 2, 3, etc. 0's to indicate the first, second, third, etc. states. We will similarly number the input symbols and directions:

$$\begin{aligned} 0 &= 0 & R &= 0 \\ 1 &= 00 & L &= 00 \\ \# &= 000 & \text{HALT} &= 000 \\ b &= 0000 \end{aligned}$$

We will use a single 1 to separate each of the five terms in a transition function, and a pair of 1's to separate the transition functions. Three consecutive 1's will denote the beginning and end of the program string.

In the Universal Language we have just described, the transition function $(q_0, 1) \rightarrow (q_1, b, R)$ is written 01001001000010, and the program string is a listing of all of the transition functions written according to the requirements of the Universal Language. The order in which the transition functions appear on the program tape is irrelevant. Below is a complete statement of the program for recognizing $0^n 1^*$ with the transition functions written in the same sequence as they appear in the listing of figure 14.20.

```
111010000101000010011010100001010001101001001000010110100010001000
101100100001001000010110010101000010011001001000100100011001000100
001000100011000100001000100001011000101000010100011000100100010010
110001000100010001000111
```

Only two symbols are used to convey all of the program information and there is no limit on the number of states that a UTM can process. The UTM need only be able to match the string written on the scratch tape with the prefix of one of the transition functions on the program tape, and this ability to test for a match will not be dependent upon the specific form of the strings being compared.

In figure 14.23, the scratch tape indicates that the problem-specific TM that is being simulated is in state q_0 and the data tape head is reading a blank. The UTM will now search the program tape for a match with the string on the scratch tape. The program head will start at the beginning of the program (the first 0 after the initial 111 end marker) and compare the symbols read character-by-character with the symbols read

by the scratch tape head. If a mismatch occurs before the scratch tape string has been completely read, the program tape moves past the first pair of 1's it observes to the start of the next transition function and the scratch tape head moves back to the start of the scratch tape string to begin the comparisons again. When a match is found, the new state is written on the scratch tape; the new symbol is written on the data tape; the data tape head is moved in the indicated direction; the code for the new symbol under the data head is appended to the scratch tape; and the comparison process between the scratch and program tapes begins again. The program terminates when a halt is located on the program tape.

Exercises

1. For the TM that recognizes palindromes described in this section indicate the elements for each of the sets Q , Γ , Σ , and F .
2. Construct a Turing machine that recognizes well-formed strings of curly brackets.
3. Construct a Turing machine that copies the contents of its tape immediately to the right of original source. Assume $\Sigma = \{0, 1, \#\}$ where $\#$ is the end of string marker, and assume the tape head is initially positioned immediately to the left of the source string.
4. Construct a one-tape Turing machine for finding the first occurrence of a target substring in a source string. Assume both the source and target are defined over a binary alphabet of $\{0, 1\}$ and use $\#$ as an end of string marker. If the target string occurs within the source, the Turing machine you design should halt with the tape head positioned over the location in the string where the target substring begins. If the target does not occur in the source string, the TM should halt with the tape head positioned over the end marker of the source. You may add symbols to the tape alphabet as needed for marking the portions of the source and target that have already been examined during the operation of this algorithm. The initial state of the TM is q_0 and the tape head is initially positioned to the left of the source string as shown in figure 14.24 below.

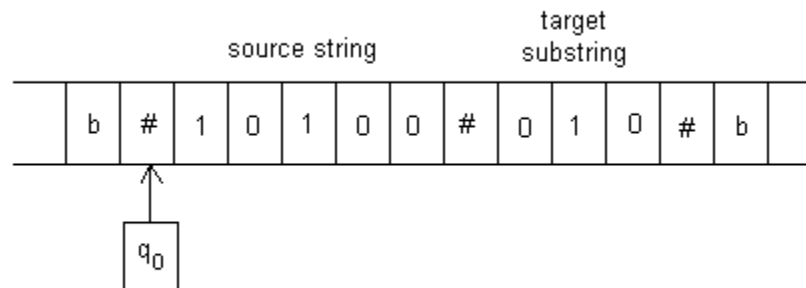


Figure 14.24 Tape input for problem 4

5. Little English is a context free language with a finite vocabulary. Each word in Little English has only one form (belongs to a single part of speech). Construct a PDA for recognizing well-formed Little English Sentences. (See chapter 1 for details.)
6. Construct a Turing machine that adds two binary integers. The tape alphabet will consist of $\{0, 1, \#, b\}$, with the $\#$ symbol separating the two terms. The tape is blank to the left and right of the input. Start with the tape head to the right of the second term. You may erase or overwrite the two input terms.
7. Express the following formulae in postfix notation
 - a. $x = ((a + b) \times (c + d) + 2 \times a) / (a + b + c)$
 - b. $x = (((a - b) \times (c + d) - (a + b)^2 \times (c + d)) \times (a^2 + b^2) \times (a + c)) / 2$
8. Implement the expression evaluation program described in section 14.4. You will need to add a class PostfixEval that reads a register containing the tokenized postfix expression and produces a numerical result. The Evaluator object must now contain both a PostfixConverter and a PostfixEvaluator. Test your program on some example expressions such as the quadratic formula.
9. Describe the algorithm for converting an infix logical expression to postfix.

14.4. Turing machines and Algorithms

A Universal Turing machine has the capability to solve any problem that can be solved on a digital computer. While it does not provide a viable blueprint for today's computer architects, the Turing machine does provide a useful vehicle for studying algorithms. The Turing machine is a standard fixed platform with finite control that reduces an algorithm to the set of mechanistic steps performed in its evaluation. We will use the Turing machine formalism to investigate questions of computability.

In the vocabulary of Turing machines we speak of recognizing a language. The Turing machine for recognizing palindromes formed over an input alphabet Σ will eventually halt in either an accepting or rejecting state. The Turing machine that performs integer addition recognizes well formed pairs of input values and halts with the sum written in a designated portion of the tape. A language L , defined over an alphabet Σ , is said to be decidable if there is a Turing machine that eventually halts in either an accepting or rejecting state for any string contained in Σ^* , the set of all possible strings that can be formed from characters in this alphabet.

In our discussion of computability, we will confine our interest to decision problems. This is not a big limitation since most problems can be reformulated as decision problems. Even an optimization problem like the traveling salesperson problem (TSP), in which we have to find a minimum distance tour of all n cities in a salesperson's territory starting and ending in the same city and visiting each other city exactly once, can be recast as a decision problem – (is there a TSP tour whose total length is less than or equal to a constant c ?). A decision problem is computable if and only if there is an

algorithmic solution that when implemented as a Turing machine will always halt in either an accepting (yes) or rejecting (no) state.

There are some decision problems for which there is no algorithmic solution. The most famous of these is the Halting Problem. The Halting problem asks the question, "Is there an algorithm for determining whether a Turing machine will halt for a particular input string?" If such an algorithm were to exist, we could construct a Turing machine that would take as its input a description of a problem-specific Turing machine and the contents of its tape and would eventually halt in either an accepting or rejecting state. This Halting algorithm would have to be able to decide yes or no for every machine and input tape pair with which it is presented.

The Halting Problem

The problem of deciding whether a given Turing machine, M , will eventually halt for a particular input, t , is not computable. No Turing machine that takes as its input a description of a machine M and its input tape t can be designed that will **halt** in an accepting state if machine M halts and in a rejecting state if machine M does not halt.

Proof

Let us suppose that a Turing machine does exist that decides whether a machine M halts for every input pair (d_M, t) , where d_M is a description of M and t is the tape input. Let us call this machine A . If machine A exists, we can design a modified Turing machine A' that halts in a rejecting state when M does not halt, but goes into an infinite loop when M halts. This modified machine, A' , can be incorporated into a Turing machine that takes a description of machine M , (d_M) , as an input and first copies this description onto its tape and then presents the input (d_M, d_M) to the modified machine A' . Call this second Turing machine B . Machine B is depicted in figure 14.25(b) below. Since M can be any Turing machine, we will let $M = B$. Machine B takes as an input its own description and halts if B does not halt or loops forever if B halts. This is obviously inconsistent, and the only step in our argument that contains a fallacy is the initial assumption. The assumption that machine A exists results in an absurdity; therefore, we have proved that a machine that decides whether a Turing machine halts for a given input cannot exist. The Halting Problem is therefore undecidable – there is no algorithm for determining whether a particular instance of any decision problem is decidable.

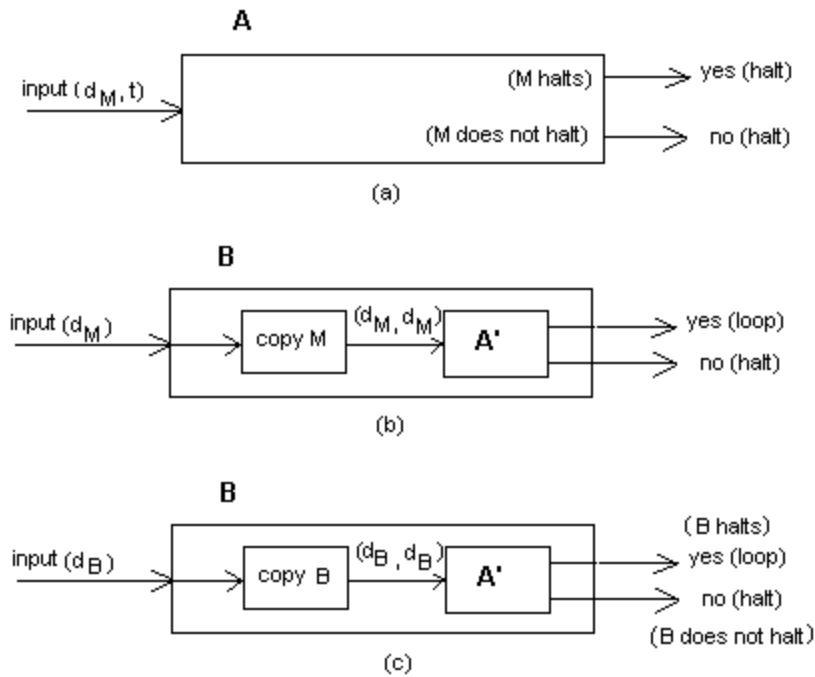


Figure 14.25. The Halting Problem

The inability to decide whether a problem-specific Turing machine will halt for a particular input string precludes the usefulness of an exhaustive search for deciding such questions as whether or not there are a set of integer assignments for the variables x , y , and z such that $x^n + y^n = z^n$ for $n \geq 3$. As long as the machine continues to run, we can never be sure whether it is because no such set of integers exists or just that we have yet to find one.

Even when we are certain that a problem can be decided with its Turing machine eventually halting in a yes or no state, the word *eventually* may render the algorithmic solution impractical. Consider the traveling salesperson problem. Suppose that a super salesman named Willy Loman has a territory of N cities with N very large, and suppose that from each city there are at least two direct routes to one of the other cities in Willy's territory. If we were to exhaustively check all of the possible routes until we found one that was less than our target value, in the worst case we would have to examine all of the more than 2^N possibilities. If N were 10,000 (and Willy was really Clark Kent assuming yet another identity) the number of possibilities, S , becomes

$$S \geq 2^{10000} = (2^{10})^4.$$

If we use the approximation $2^{10} = 1024 \approx 10^3$, then $S \geq 10^{12}$. If we assume that each computation requires 100 microseconds, it could take at least 3 years of continuous computation before we know whether or not a route that meets our target criteria exists. Willy could be dead by that time!

If we cannot find a better algorithm than exhaustive search that works for every instance of TSP, then the problem is intractable. There are fast algorithms that give a reasonably close estimate of the optimal tour length, but, to date, we do not know of an algorithm other than exhaustive search that decides every instance of TSP.

When we speak of a fast algorithm in the previous paragraph, we mean one whose computation time is $O(P(N))$ where $P(N)$ is polynomial in the number of cities N . If, for instance, we find an algorithm whose number of computations is less than or equal to N^3 and the time per calculation were again 100 microseconds, the total computation time would be reduced from over 3 years to under 2 weeks. Problems of size n for which there is an algorithmic solution whose computation time is $O(P(n))$ are said to belong to the complexity class $\mathcal{P}$.

While there is no known polynomial-time algorithm for deciding all instances of TSP, we are able to verify that a candidate solution for a particular problem instance is correct in polynomial-time. Given a graph depicting the cities in the territory and their connecting links and a sequence of cities that constitute our suggested tour, we would need to verify that each of the N cities in the territory occurs once and only once in the tour (except for the starting and ending point), that a path exists between each pair of cities in the tour, and that the total path length of the tour is less than or equal to the target value. All of these steps can be performed in polynomial time.

Of course this candidate solution would have had to arise from either magic or a lucky guess. We might say that we have magical dice that are rolled every time we have to make a decision about which path to take, and every time lead us to make the right choice. Such dice would not only magically produce a solution, but that solution would be generated in a time that was $O(N)$.

Problems for which a candidate solution can be verified in polynomial-time are said to belong to the class $\mathcal{NP}$. The term $\mathcal{NP}$ stands for non-deterministic polynomial, and problems in class $\mathcal{NP}$ can be solved in polynomial-time on a non-deterministic Turing machine. The non-deterministic Turing machine (NTM) can be understood in terms of our magical dice. The tour path is not generated in some deterministic manner on an NTM, but at each transition of the machine, the new state, tape symbol, and move is selected arbitrarily from a finite set of choices. Each transition of the NTM behaves similarly to a roll of the dice with a fortuitous choice based solely on "machine intuition". The NTM will not accept a string that is not part of its language, but the string (path) that is accepted is any one of the feasible choices. Both the NTM and the dice are able to work their magic in polynomial-time.

If we are not fortunate enough to own magic dice but do have access to a large distributed network of computers, we are still able to produce a solution to TSP in polynomial-time. Suppose we are able to employ all of the active but idle capacity of the Internet. This vast, but not infinite, resource will simulate an NTM with a possibly insufficient supply of tape to handle a large-sized problem.

We begin solving this problem on a single machine, but when an m -way branching decision needs to be made we find $m-1$ idle computers and send them a copy of the problem statement and a copy of the partial path determined so far. Each newly impressed machine is instructed to take one of the $m-1$ branches, and the original machine chooses the remaining one. Every time another branching decision has to be made by one of the active machines, the process is repeated. The number of machines in

use grows exponentially, but the time for any one of them to complete is bound by a polynomial function of N . When all of the active machines have completed their processing, any finding a solution less than the target value will relay a message to the machine that first contacted it. A solution propagates back to the original machine after N passes.

Any problem that can be solved in polynomial-time can be verified in polynomial-time. We need only supply a deterministic Turing machine with a description of the problem and a certificate containing the candidate solution. To verify the certificate the TM solves the problem and compares the two solutions. The class $\mathcal{P}$ therefore is a subclass of $\mathcal{NP}$.

The question of whether any problem that can be verified in polynomial-time can also be solved in polynomial-time is open. If true, we have $\mathcal{P} = \mathcal{NP}$. The two classes would be equivalent. While it is generally believed that $\mathcal{P} \neq \mathcal{NP}$, the two classes are not equivalent, a proof has not been found. Significant work has been done, however, in defining the approach that a proof about the equivalence of these two classes would likely take. The study of this fundamental question has deepened our understanding of algorithms and engaged computer scientists, mathematicians, and physicists in an interdisciplinary study of the fundamental aspects of computation.

Exercises

1. Write a program to verify that a candidate solution for a TSP tour is correct. The input for this program will consist of a graph, with the cities as the vertices and the connecting roads as edges, and the certificate. The graph may be expressed as an $N \times N$ matrix with the $(i, j)^{\text{th}}$ entry indicating the distance between the cities numbered i and j . If no road directly connects this pair of cities, the matrix entry will be 0. The certificate consists of a sequential list of the cities visited during the tour and the target value that the tour is not to exceed. Demonstrate that your program decides this question in $O(P(N))$ time.

14.5. Summary and Conclusion

In this chapter we have considered several different machines with processing capability. The finite state machine has a finite set of internal states and responds to a sequence of external events. It lacks the full capability of a computer because it lacks an extensible memory. The push down automata adds a stack to its finite control. It is able to make a transition based upon the current input symbol and the token on the top of the stack. The push down automata is capable of recognizing well-formed sentences in certain context free languages, such as those used in programming, but is not able to . The Turing machine is equipped with an infinitely long tape that provides the equivalent of random access storage. The tape head is able to move in both directions and is capable of reading and writing symbols on the tape. The Turing machine has all of the capabilities of a random access computer and provides a platform for the formal study of algorithms.

Using a Turing machine formalism we are able to define three classes of decision problems. The first is the class of problems that are computable. For all problems in this

class a Turing machine can be designed that will always eventually halt in either an accepting or rejecting state. The second class may be described as partially computable. For problems in this class, a Turing machine can be constructed that will always eventually halt in an accepting state when the answer is "yes", but may not halt if the answer is "no". The third class of problems are those that are unsolvable. No Turing machine exists that, when applied to a problem in this class, always eventually terminates in an accepting state. The Halting problem is one important example of an unsolvable problem.

The phrase "eventually terminates" leads to an additional classification. Problems for which there is a Turing machine that will always terminate in a time that is a polynomial function of the size of the problem are said to belong to the class $\mathcal{P}$. These are the problems for which a computer solution is practical. For any problem not in class $\mathcal{P}$ there is no known algorithm that can always produce a solution in polynomial time. Problems not in class $\mathcal{P}$ are said to be intractable because the amount of time to find a solution to most instances of these problems far exceeds any practical limit such as the lifetime of the programmer. For many of these problems, however, polynomial-time algorithms that give approximate solutions do exist.

An important subclass of the non-polynomial-time algorithms are those that can be verified in polynomial-time. These algorithms are said to belong to the class $\mathcal{NP}$, and play a pivotal role in the understanding of computer algorithms. Since every problem in $\mathcal{P}$ can also be verified in polynomial-time we have $\mathcal{P} \subset \mathcal{NP}$. It is an open question whether all problems that can be verified in polynomial time can also be solved in polynomial time and hence whether $\mathcal{P} = \mathcal{NP}$.